Appendix

Anonymous Author(s)

ACM Reference Format:
Anonymous Author(s). 2026. Appendix. In *Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026), Paphos, Cyprus, May 25 – 29, 2026*, IFAAMAS, 3 pages.

1 APPENDIX A – ADAPT METHOD TO NEW ENVIRONMENT

To use this model in any environment, collect expert data (e.g., using IPPO[1] for a specific task) containing trajectories with observation, action, and reward information. The observation is expected to be a vector. In vector environments, the meaning of each element's value is usually known.

Manually create a positional encoder for the environment. For each observation element obs[i], assign positional numbers as follows:

- time[i]: indicates the time step (0 = current, 1 = previous, etc.). For example, if the observation includes the last five actions of one agent, these elements have time encodings 0 to 4, while agent, group, and attribute encodings remain the same.
- agent[i]: specifies the agent related to the information. For example, agents can be numerated $(1, 2, \dots)$ in the observation, and information assigned to the main agent or environment can be numerated by 0.
- group[i]: indicates the agent's group (0 if there is only one group). For instance, groups can represent allies (0) and enemies (1).
- identifies the feature type; equivalent features across agents or time share the same attribute index, otherwise they are numbered sequentially $(0, 1, 2, \dots)$. For example, observations may include the health points (hp) of each visible agent, and these elements correspond to different agents but share the same attribute index.

The final step is to adapt the general dataloader to the new environment using the created positional encoder.

2 APPENDIX B – IMPLEMENTATION DETAILS

The open-source repository provides all resources needed to reproduce MARL-GPT results, including scripts for dataset generation, training, and evaluation. Example scripts and detailed Docker setup instructions are included to help users get started quickly, along with configuration files for benchmarks and datasets to ensure experiments can be easily replicated.

The MARL-GPT code is based on NanoGPT[2] framework. This framework was selected for its straightforward and modular design, allowing for easy modification and adaptation. Training the 7M-parameter model in the main experiment for 500K iterations took 161 hours using 8 NVIDIA H100 80GB GPUs. In other experiments, training the same model for 10K iterations took 13 hours on 2 NVIDIA H100 80GB GPUs.

Parameter	Value
Max size of observation	700
Action size	20
Number critic bins for each action	20
Max value	5
Number sampled actions for critic regularization	4
α_{td}	1
α_{cr}	1
α_a	0.1
α_{bc}	1
history length (for SMACv2 and GRF)	6
γ	0.95
Target Network Update	0.7
Minimum learning rate	6e-5
Maximum learning rate	6e-4
Learning rate decay	cosine
AdamW optimizer beta1	0.9
AdamW optimizer beta2	0.95
Gradient clipping	1.0
Weight decay	1e-1
Warm-up iterations	2000
Use PyTorch 2.0 compilation	True
Gradient accumulation steps	16
Total training iterations	500,000
Batch size	900
Number of layers	8
Number of attention heads	8
Embedding size	256

Table 1: Learning Hyperparameter Details.

Table 1 provides the list of hyperparameters used in our main experiments.

When the model was fine-tuned on 10-agent SMACv2 (chapter 5.2.3) data mixed with batches from original tasks to prevent catastrophic forgetting, so no performance drop occurred.

3 APPENDIX C – ADDITIONAL SINGLE-TASK COMPARING

3.1 Multi-agent baselines.

We included established multi-agent online-trained baselines for SMACv2 and GRF (Table 2), specifically IPPO, QMIX, and QPLEX. These approaches use smaller networks and require less training time, but they are tailored for single tasks and some have centralized

[1]https://github.com/alex-petrenko/sample-factory
[2]https://github.com/karpathy/nanoGPT

Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026), C. Amato, L. Dennis, V. Mascardi, J. Thangarajah (eds.), May 25 – 29, 2026, Paphos, Cyprus. © 2026 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org). This work is licenced under the Creative Commons Attribution 4.0 International (CC-BY 4.0) licence.

critic. Despite these advantages, MARL-GPT significantly outperformed them, consistent with previous reports [2, 3]. These results demonstrate that our single model can effectively handle a variety of game instances as a unified policy.

Env	Task	IPPO	QMIX	QPLEX	MARL-GPT
SMAC	protoss 5_vs_5	54.6	70.2	53.3	89
	terran 5_vs_5	56.2	58.4	70.0	93
	zerg 5_vs_5	37.2	37.2	47.8	74
GRF	pass and shoot	93.7	95.6	88.1	96
	corner	50	20	28.8	83
	counterattack easy	84.1	16.0	94.9	49

Table 2: Win rates (%) for selected methods. Comparing MARL-GPT and methods, which was trained just on one task.

train \ val	terran_5_vs_5	terran_5_vs_6	protoss_5_vs_5	protoss_5_vs_6	zerg_5_vs_5	zerg_5_vs_6
terran_5_vs_5	**79.0**	15.4	2.2	0.2	44.0	11.4
terran_5_vs_6	77.0	**38.8**	3.4	0.2	44.4	23.0
protoss_5_vs_5	1.2	0.0	**72.4**	6.4	2.6	0.8
protoss_5_vs_6	0.4	0.0	74.6	**17.6**	1.2	0.4
zerg_5_vs_5	46.8	6.8	3.0	0.0	**66.0**	20.8
zerg_5_vs_6	47.8	16.0	0.8	0.2	63.4	**33.0**

Table 3: Single-Task DT on SMAC: cross-scenario validation win-rates (%). Each DT model is trained from scratch on the scenario indicated by the row and evaluated on all six scenarios listed in the columns.

3.2 Insights from Single-Task DT

To isolate the contribution of each SMAC scenario to cross-task generalization, we trained six single-task DT agents–one per scenario–using only 2% of the available trajectories. This deliberately light-training regime seeks to *reveal* how the source scenario shapes transfer, rather than to chase the highest possible win rates. We then evaluated every agent on the full 6×6 grid of tasks; the results are summarized in Table 3 and expose two salient patterns.

First, inter-race transfer is asymmetric: a DT trained on Terran scenarios generalizes surprisingly well to Zerg tasks (e.g., 38.8 % on terran_5_vs_6 → terran_5_vs_6 and 44.4 % on terran_5_vs_6 → zerg_5_vs_5), while the converse holds for a Zerg-trained agent; by contrast, a Protoss-only agent shows almost no transfer beyond its own race. This suggests that Terran and Zerg share tactical motifs, such as kiting and surround mechanics [1], that are largely absent from Protoss micro, whose reliance on shield management and ranged focus fire produces a more domain-specific policy space. Second, curriculum matters: models trained on the harder 5 vs 6 variant outperform those trained on 5 vs 5 when validated on both difficulty levels of the same race, indicating that exposure to the richer state–action distribution of the difficult setting yields representations that subsume the easier case rather than overfitting to it.

4 APPENDIX D – ROBOTICS EXPERIMENTS

4.0.1 Maze Environment. The physical environment is designed as a modular arena composed of standardized floor tiles and wall segments, allowing for rapid reconfiguration of the maze layout. Each floor tile is a 30 cm × 30 cm square plywood module with pre-drilled holes that support the attachment of wall elements. Wall segments are lightweight and can be mounted at any angle relative to the tile grid using custom connectors. Fig. 1 illustrates the wall placement mechanism.

To accommodate different experimental scenarios, wall modules can be stacked to increase their height, providing visual occlusion or physical boundaries as needed. This design enables a wide range of maze configurations to be assembled with minimal effort.

Figure 1: Modular and reconfigurable maze environment composed of standardized floor and wall units.

4.0.2 Robotic Agents. Each agent is a custom-modified variant of the Waveshare JetBot platform, significantly enhanced to support onboard perception, localization, and control. The robot is equipped with a pair of DC motors with encoders and an RPLIDAR A1 2D laser scanner mounted on top for mapping and obstacle detection. A Jetson Nano 4,GB module running Ubuntu 20.04 serves as the primary onboard compute unit.

This is a differential-drive robot with dimensions of 17 cm × 17 cm × 22 cm (length × width × height), making it suitable for confined environments such as narrow maze corridors. Fig. 2 illustrates the robots.

All software runs inside Docker containers hosted directly on the robot. The system stack includes ROS 2 Humble, compiled from source. Each robot operates independently in an isolated namespace, with segregated ROS 2 topics and TF trees, enabling scalable multi-agent deployment without inter-agent interference.

For mapping, we employ **SLAM Toolbox**, which incrementally builds a 2D occupancy grid using onboard lidar and odometry data. Navigation is handled by the **Nav2** stack, configured for both global and local planning within the constructed map. Low-level motor control is performed using the **ros2_control** framework, integrated

Figure 2: Waveshare JetBot robotic agent used for maze navigation experiments.

with a custom hardware interface developed specifically for this platform.

4.0.3 Experimental Protocol. The physical maze was assembled to replicate the layout used in the corresponding simulation scenario. Due to a limited number of available wall segments, the walls were not placed in continuous contact with each other. To compensate for these small gaps, the SLAM configuration employed an adjusted *inflation radius* parameter, ensuring that the robot would interpret adjacent wall segments as a continuous obstacle and avoid traversing between them. As a result, the perceived map maintained topological consistency with the simulated environment.

An example of such a scenario is presented in Fig. 3, selected from several that were tested. Shown here is an instance in the POGEMA environment alongside its corresponding real-world setup. The scenario was originally evaluated in simulation and then reconstructed in the physical environment using the same initial agent positions and target locations. The MARL-GPT policy was used without modification to control the agents during these real-world trials.

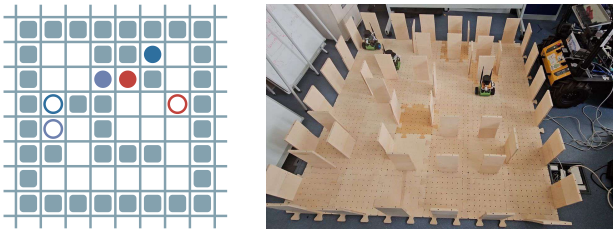

Figure 3: Real-world execution of a scenario based on maze scenario with 3 robots. Left: simulated environment in POGEMA. Right: corresponding physical setup used for real-world deployment.

The resulting multi-agent behaviors, including coordination and conflict resolution, were recorded and are presented in the accompanying video materials. In conclusion, the experiments demonstrate that the approach is effective in the real world and that the agents are capable of coordinating their actions to successfully reach their goals.

REFERENCES

[1] Benjamin Ellis, Jonathan Cook, Skander Moalla, Mikayel Samvelyan, Mingfei Sun, Anuj Mahajan, Jakob Foerster, and Shimon Whiteson. 2023. SMACv2: An Improved Benchmark for Cooperative Multi-Agent Reinforcement Learning. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 37567–37593. https://proceedings.neurips.cc/paper_files/paper/2023/file/764c18ad230f9e7bf6a77ffc2312c55e-Paper-Datasets_and_Benchmarks.pdf

[2] Tien Mai, Thanh Nguyen, et al. 2024. Mimicking To Dominate: Imitation Learning Strategies for Success in Multiagent Games. *Advances in Neural Information Processing Systems* 37 (2024), 84669–84697.

[3] Yan Song, He Jiang, Haifeng Zhang, Zheng Tian, Weinan Zhang, and Jun Wang. 2023. Boosting studies of multi-agent reinforcement learning on Google research football environment: The past, present, and future. *arXiv preprint arXiv:2309.12951* (2023).